Software Architecture and Design Specification

Collaborative Study Material and Online Quiz Platform (StudyHub)

Prepared following IEEE Std 1016-2009 (Software Design Descriptions) and ISO/IEC/IEEE 42010 viewpoints

Team Members:
Sanjith K S (PES1UG24CS613)
Fathima Zahra (PES1UG24CS664)
Dhanush S Shekhar (PES1UG24CS662)

Version 2.0 | 7 November 2026 | Describes the system as built at the end of Sprint 2

Version	Date	Description
1.0	19 Oct 2026	Initial architecture at the start of Sprint 1 (11 modules, planned web UI).
2.0	7 Nov 2026	Updated to the delivered system after Sprint 2: 14 modules, full web UI, security architecture, API design, error handling, traceability and deviations.

Contents. 1 Introduction · 2 Stakeholders and design concerns · 3 Architectural representation · 4 Goals and constraints · 5 Architectural pattern · 6 Context view · 7 Component view · 8 Data view · 9 Interaction view (sequence diagrams) · 10 API design · 11 Error handling protocols · 12 Security architecture · 13 Deployment view · 14 Quality attributes · 15 Traceability · 16 Deviations from the SRS and the initial SAD · Appendix A Module dependency matrix

1. Introduction

1.1 Purpose

This Software Architecture and Design (SAD) specification describes the architecture and the detailed design of StudyHub, the Collaborative Study Material and Online Quiz Platform, as delivered at the end of Sprint 2 (30 October 2026). It explains how the system is structured, why it is structured that way, how its parts interact, how it is secured and deployed, and how every requirement in the SRS is realised. It is written for the development team, the course evaluators and anyone who maintains the system.

1.2 Scope

The document covers the whole system: the browser-based web UI, the FastAPI application server with its fourteen feature modules, the PostgreSQL database, the external LLM service and the GitHub-based build and deployment pipeline. It covers functional requirements FR-01 to FR-14, non-functional requirements NFR-01 to NFR-07 and security requirements SEC-01 to SEC-13. Where the delivered system differs from the SRS or from version 1.0 of this document, the difference is recorded in section 16.

1.3 Definitions, acronyms and abbreviations

Term	Meaning
API	The REST/JSON application programming interface under /api/v1.
Module	A Python package under app/modules/ that owns its database tables and exposes one public interface, service.py.
Service interface	The functions and data classes in a module's service.py. Other modules may call only these.
Capability	A single permitted action such as material:upload or user:manage. Routes are guarded by capability, not by role name.
Domain event	An in-process notification (for example ATTEMPT_SCORED) that lets one module react to another without importing it.
ORM / migration	SQLAlchemy 2 object-relational mapping; Alembic manages versioned, reversible schema migrations.
LLM	Large Language Model: the external AI service used for question generation, answer evaluation and explanations.
JWT / HS256	JSON Web Token signed with HMAC-SHA256, used as the bearer access token.
CI	Continuous Integration: GitHub Actions job lint-and-test, required before any merge into main.
CSP	Content Security Policy: a browser header restricting where a page may load code and data from.
FR / NFR / SEC	Functional, non-functional and security requirement identifiers from the SRS and the Security Requirements Specification.

1.4 References

- IEEE Std 1016-2009, IEEE Standard for Information Technology: Software Design Descriptions.
- ISO/IEC/IEEE 42010:2011, Systems and software engineering: Architecture description.
- SRS Document (Documents/SRS Document.docx) and Use Case Flow / Use Case Diagram (Documents/UseCaseFlow.docx, UseCaseDiagram.drawio).
- Test Plan Document (Documents/Test Plan Document.docx) and the updated test case document and reports in docs/.
- Security Requirements Specification (Documents/Security_Requirements_Document.pdf).

- Repository: README.md, docs/architecture.md, docs/module-interfaces.md, docs/api/openapi-v1.json, docs/performance.md, docs/test-execution-log.md.

1.5 Overview

Sections 2 to 5 set out the stakeholders, the viewpoints used, the goals and constraints and the chosen architectural pattern. Sections 6 to 13 describe the architecture view by view: context, components, data, interactions, API, error handling, security and deployment. Section 14 shows how the quality attributes are met, section 15 traces requirements to design and tests, and section 16 records deviations. Every diagram has its own legend that defines its boxes and lines.

2. Stakeholders and design concerns

Stakeholder	Main concerns	Addressed in
Students	Find materials and quizzes quickly; fair, instant scoring; privacy of their data	6, 9, 12, 14
Authorized users (faculty, mentors)	Publish official content; control AI-generated questions before students see them	7, 9.3, 12
Administrators and moderators	Approve channels, act on reports, suspend users, audit actions	7, 12, 15
Development team (3 members)	Work in parallel on separate modules; change one module without breaking others; fast feedback from CI	5, 7.3, 13, 14
Course evaluators	Standard format, traceability to the SRS, justified architectural choices, test evidence	3, 5, 15, 16
Operators	Simple deployment, no secrets in the repository, recovery after failures	11, 13

3. Architectural representation

The architecture is described through the IEEE 1016 / ISO 42010 viewpoints below. Each diagram fits on one page, decomposes the system hierarchically and carries its own legend.

Viewpoint	What it shows	Notation	Section
Context	System boundary, users and external systems	Context diagram	6
Composition (logical)	The three layers and every component in them; module dependencies	UML component diagram; dependency graph	7
Information	Tables, keys, relationships and owning module	Entity-relationship diagram	8
Interaction	Run-time collaboration for key scenarios	UML sequence diagrams	9
Interface	Endpoints, conventions, status codes, contract	Tables; OpenAPI 3 document	10, 11
Security	Trust boundaries and controls on the request path	Security architecture diagram	12
Deployment	Nodes, containers, storage, CI/CD	UML deployment-style diagram	13

4. Architectural goals and constraints

4.1 Goals

Goal	Source	How the architecture meets it
Fast common operations: pages and search within 2 s, quiz submission within 3 s at the 95th percentile with 50 concurrent users	NFR-01	Stateless API, indexed query paths, admission control sized to the connection pool, in-process deterministic scoring.
Credentials protected and access enforced on the server	NFR-02, SEC-01..13	Six-stage request path: admission, validation, authentication, capability and membership checks, service rules, response schemas (section 12).
Intuitive navigation and visual analytics	NFR-03	Dashboard landing page, one-click navigation, SVG charts; the UI holds no rules of its own.
No committed attempt or upload is ever lost	NFR-04	One transaction per request, answers autosaved, domain-event handlers isolated in savepoints, transient failures reported as 503 with nothing half-written.
Performance holds as data and users grow	NFR-05	Work per request independent of data volume, hot paths indexed, stateless servers can be replicated.
Trustworthy AI content	NFR-06	Review gate for generated questions, validation of every LLM answer, reporting of AI output, scoring never uses the LLM.
Change one feature without touching others	NFR-07	Modules own their data and expose one interface; dependency rules enforced by an automated test; documented interfaces.

4.2 Constraints

- **Team and schedule:** three developers, two one-week sprints (19–23 and 26–30 October 2026), one pull request per story.
- **Technology:** Python 3.12, FastAPI, SQLAlchemy 2, Alembic, PostgreSQL 16 (SQLite allowed for local development), plain HTML/CSS/JavaScript, Docker.
- **Process:** main is protected; a pull request merges only after the lint-and-test check passes.
- **Security:** no secret in the repository; all secrets come from environment variables.
- **External dependency:** the LLM is a third-party service with variable latency; the platform must keep working when it is slow, failing or switched off.

5. Architectural pattern and rationale

Chosen style. StudyHub combines four patterns:

- **Client-server:** browsers are thin clients; the server holds all business rules, security checks and data.
- **Layered:** user interface, frontend logic and backend. Inside the backend, the layers are API, services, data access and infrastructure, and each layer calls only the layers below it.
- **Modular monolith:** one deployable process split into fourteen feature modules with enforced boundaries.
- **In-process event-driven integration:** a module publishes a domain event and subscribers react in the same transaction without being imported.

Alternative	Why it was not chosen
Microservices (one service per feature)	Network calls, distributed transactions and separate deployments would cost a three-person team more than the two sprints allow, and would make NFR-04 (no lost attempts) much harder. The module boundaries keep this option open: any module can be extracted later because it already talks to others only through its service interface.
Classic MVC monolith without module boundaries	Fast to start, but every feature could reach every table, so a change to chat could break quiz scoring. That fails NFR-07 and the SRS requirement that modules be independently maintainable.
Server-rendered pages (templates)	A separate static frontend over a documented API lets the UI and API be built and tested in parallel and keeps every rule on the server.

Consequences. Low coupling: modules exchange plain data classes, never ORM objects. High cohesion: each module groups one feature's rules and tables. Shared transactions make reliability simple. The cost is that the whole application scales as one unit. That is acceptable at the measured load: 50 concurrent users stay well inside target (section 14).

6. Context view

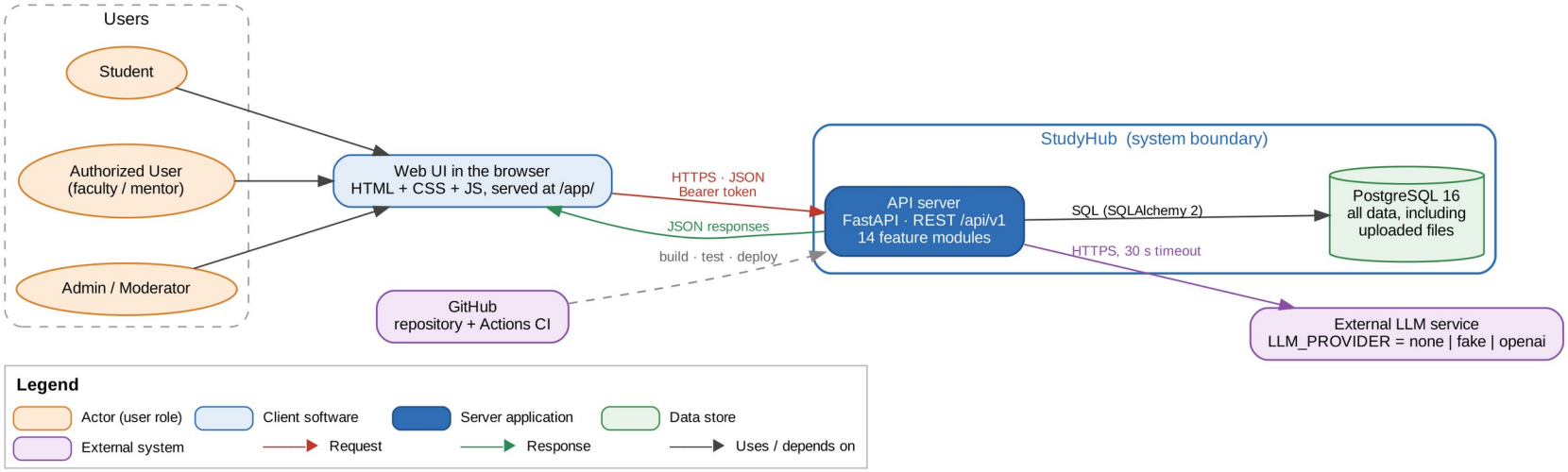


Figure 1: Context: users, the system boundary and external systems

Three kinds of users reach StudyHub through the web UI, which the API server itself serves at /app/. The browser sends HTTPS requests with a JSON body and a bearer token. The API server is the only component that touches the database or the LLM service. Uploaded files are stored in PostgreSQL, so every API instance sees the same files. The LLM provider is chosen by configuration (none, fake for offline demos and tests, or an OpenAI-compatible service), and the rest of the platform keeps working when it is unavailable.

7. Component view

7.1 Component diagram

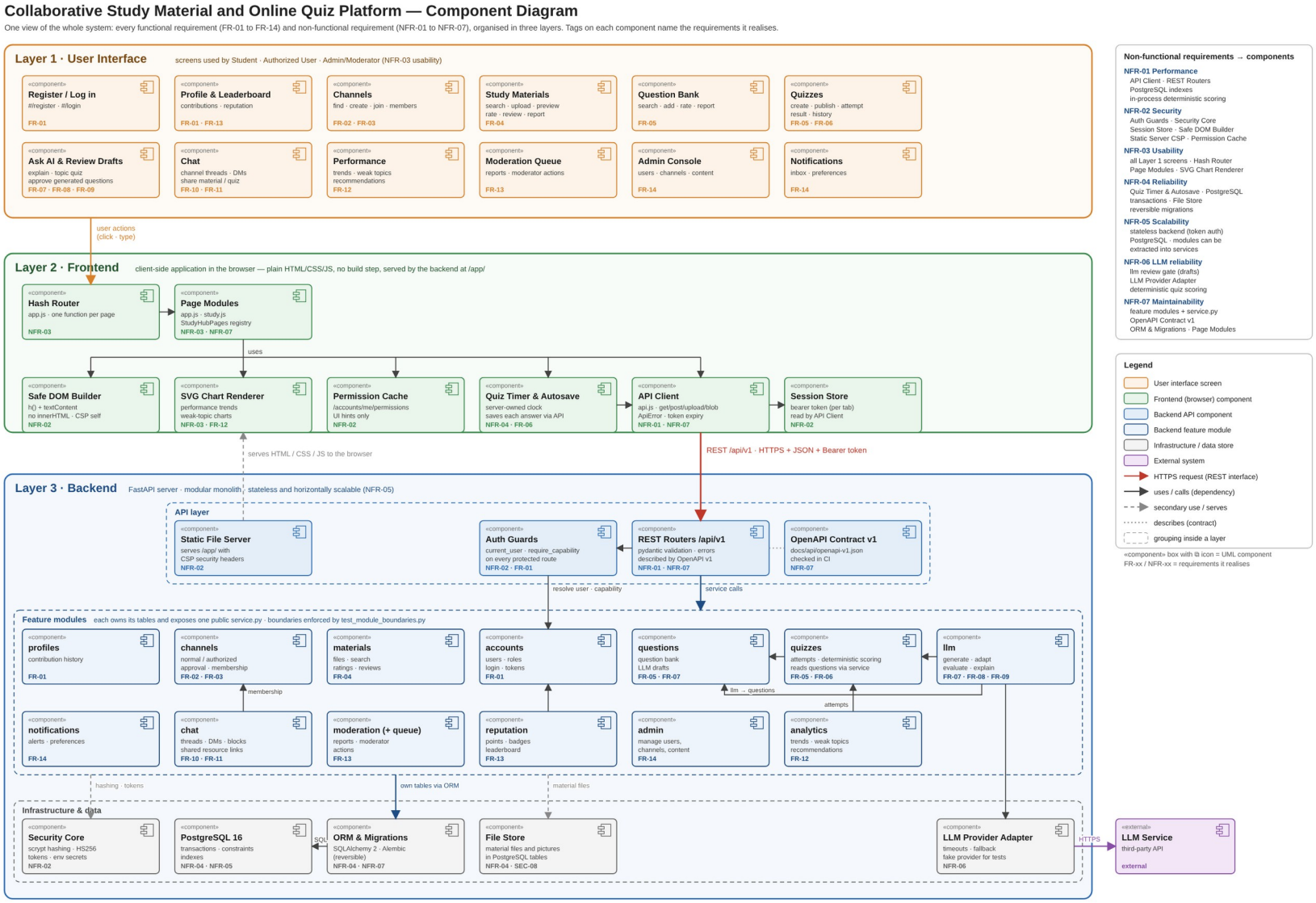


Figure 2: UML component diagram: user interface, frontend and backend layers with requirement tags

7.2 Component descriptions

Layer	Component	Responsibility	Requirements
User interface	Screens	Register/log in, dashboard, channels, materials, question bank, quizzes and attempts, quiz history, Ask AI and draft review, performance and recommendations, chat and direct messages, leaderboard, notifications, moderation queue, admin console.	FR-01..FR-14, NFR-03
Frontend	Hash Router, Page Modules	app.js and study.js map each #/route to a page function that builds the screen from API responses.	NFR-03, NFR-07
Frontend	API Client, Session Store	api.js wraps fetch with the bearer token, JSON and file uploads, and turns error responses into ApiError messages. The token is kept per browser tab.	NFR-01, NFR-02
Frontend	Safe DOM Builder, SVG Charts, Quiz Timer, Permission Cache	Builds elements with textContent only (no innerHTML), draws analytics charts, shows the server-owned countdown and autosaves answers, and caches capabilities to hide controls the user cannot use (the server still enforces them).	NFR-02, NFR-03, NFR-04, FR-12
Backend: API	REST routers, validation, error handlers	18 routers under /api/v1. Pydantic schemas validate every request, and uniform handlers produce the error format in section 11.	NFR-07, SEC-07
Backend: API	Auth guards, RequestLimit, static server	current_user and require_capability on protected routes; admission control sized to the database pool; /app/ served with CSP and nosniff headers.	NFR-01, NFR-02
Backend: services	14 feature modules	Business rules per feature. See the table below and Appendix A.	FR-01..FR-14
Backend: infrastructure	Database, ORM and migrations, security core, events, LLM provider	PostgreSQL via SQLAlchemy sessions (one per request), Alembic migrations, scrypt hashing and HS256 tokens, in-process domain events with savepoints, LLM adapter with a 30 s timeout.	NFR-02, NFR-04, NFR-06

Module	Responsibility	Owns tables	FR
accounts	Registration, login, tokens, roles and capabilities, route guards, profile pictures	users, profile_pictures	FR-01
profiles	Profile page with contribution history	-	FR-01
channels	Normal and Authorized channels, approval, membership, discussion posts	channels, channel_memberships, channel_posts	FR-02, FR-03
materials	Upload, search, preview, download, rating and reviews of study materials	study_materials, material_files, material_reviews	FR-04
questions	Question bank: contribute, search, rate, report	questions, question_ratings	FR-05
quizzes	Quiz assembly and publishing, attempts, timer, deterministic scoring, history	quizzes, quiz_questions, quiz_attempts, attempt_answers	FR-05, FR-06
llm	Question generation with review, adaptive difficulty, descriptive evaluation, Ask AI	llm_requests, adaptive_sessions	FR-07, FR-08, FR-09
chat	Channel chat with threads, direct messages, blocking, shared resources	channel_messages, conversations, conversation_participants, direct_messages, user_blocks	FR-10, FR-11

Module	Responsibility	Owns tables	FR
analytics	Performance by subject and topic, strengths and weak areas, recommendations	performance_attempts, performance_topics, recommendation_opens	FR-12
reputation	Reputation points, badges, leaderboard	reputation_events, badges, user_badges	FR-13
moderation	Reports and the moderation action log	reports, moderation_actions	FR-13, FR-14
moderation_queue	Assembles the moderation queue from the modules that own the content	-	FR-13
notifications	In-app notifications and per-category preferences	notifications, notification_preferences	FR-14
admin	Admin dashboard, user suspension, channel approval, content removal	-	FR-14

7.3 Module dependencies

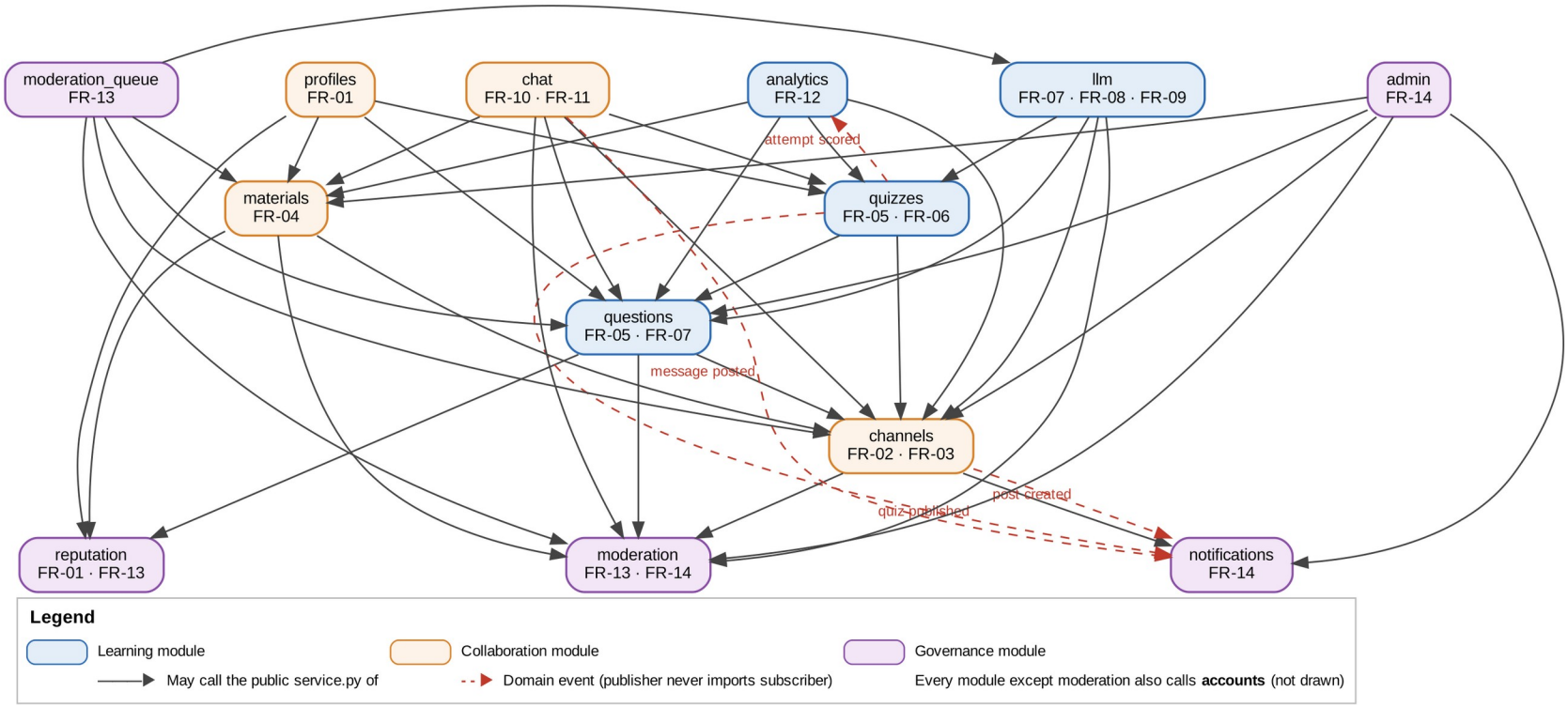


Figure 3: Module dependency view: allowed service calls and domain events

Rules: a module may import app.core, its own package and the service.py of the modules listed for it in ALLOWED_DEPENDENCIES (Appendix A). It never imports another module's models. Reactions that would create a cycle use domain events instead: quizzes publishes ATTEMPT_SCORED and analytics subscribes; quizzes, chat and channels publish events that notifications subscribes to. tests/test_module_boundaries.py fails the build if a rule is broken, and docs/module-interfaces.md is generated from the code so the documented interfaces cannot drift.

8. Data view

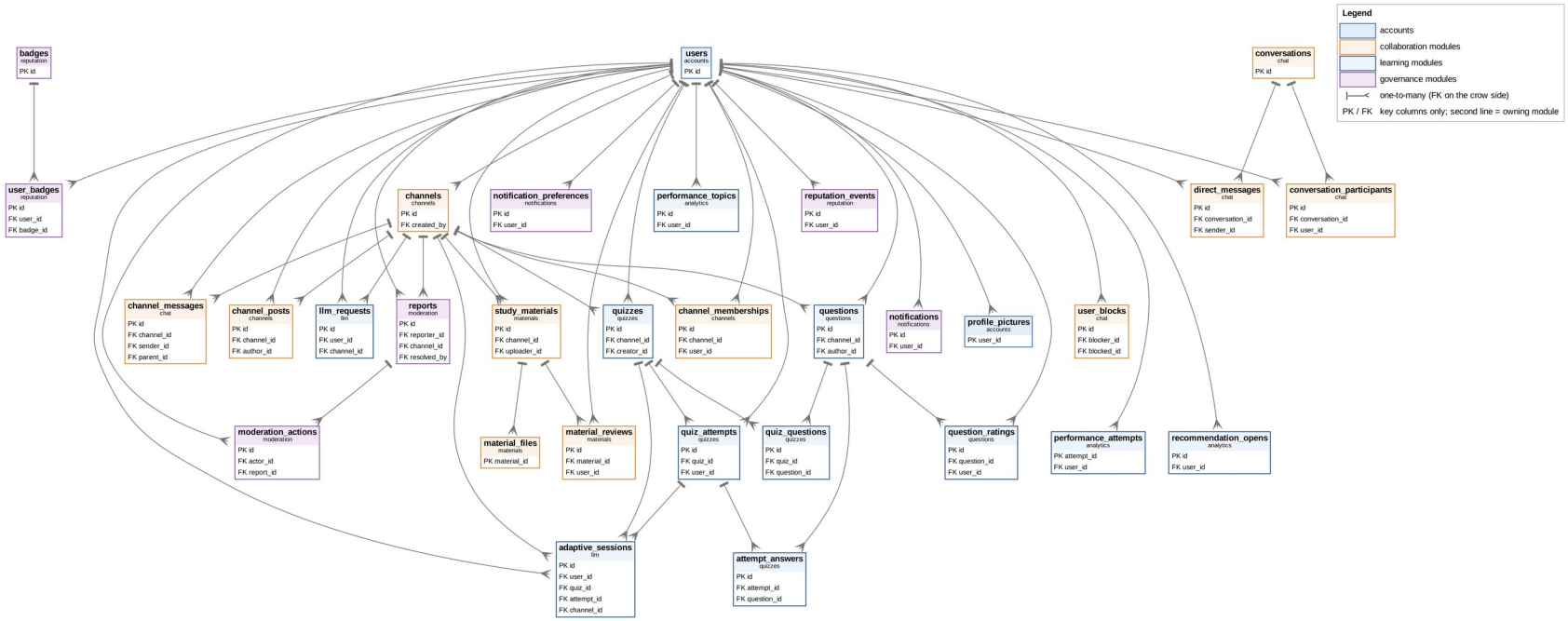


Figure 4: Entity-relationship diagram generated from the ORM metadata (31 tables, 48 foreign keys)

All data lives in one PostgreSQL database, but every table belongs to exactly one module. Foreign keys keep the data consistent across modules without Python imports between them. Links that can point at any kind of content, such as a report target or a resource shared in chat, are stored as a type plus an id. Enumerations are stored as VARCHAR with CHECK constraints so the same migrations run on PostgreSQL and SQLite. Uploaded files and profile pictures are stored in tables with deferred loading, so listings never read file bytes. Schema changes are Alembic migrations that are tested to upgrade and downgrade cleanly on both databases.

9. Interaction view: sequence diagrams

Solid arrows are calls or requests, dashed arrows are returns, numbers give the order, and alt and loop frames show alternatives and repetition. Notes state the rule the scenario protects.

9.1 Log in and open a protected resource (FR-01, FR-02, SEC-03..06)

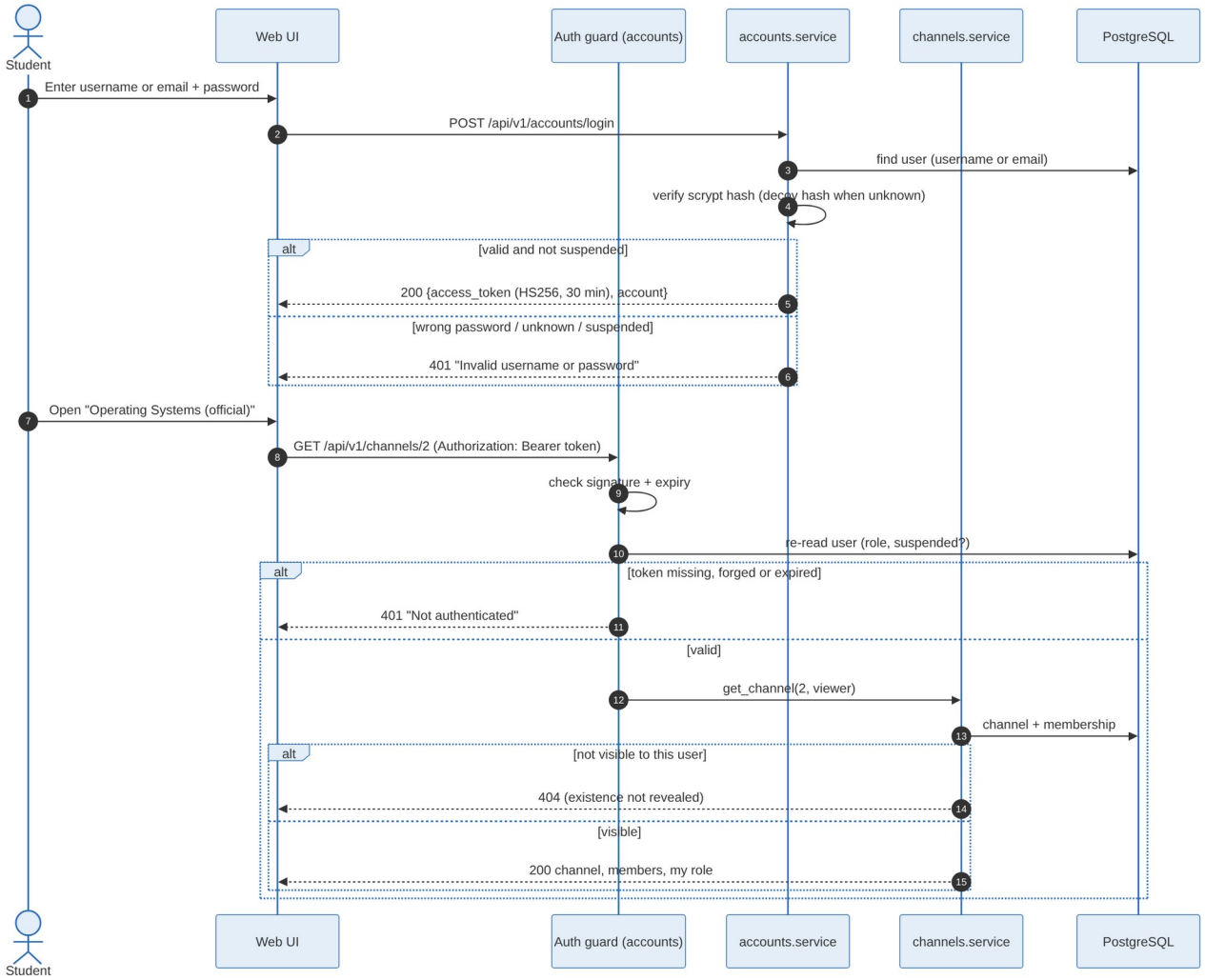


Figure 5: Sequence: login, token check, capability and visibility check

9.2 Attempt, submit and score a quiz (FR-06, FR-08, NFR-01, NFR-04)

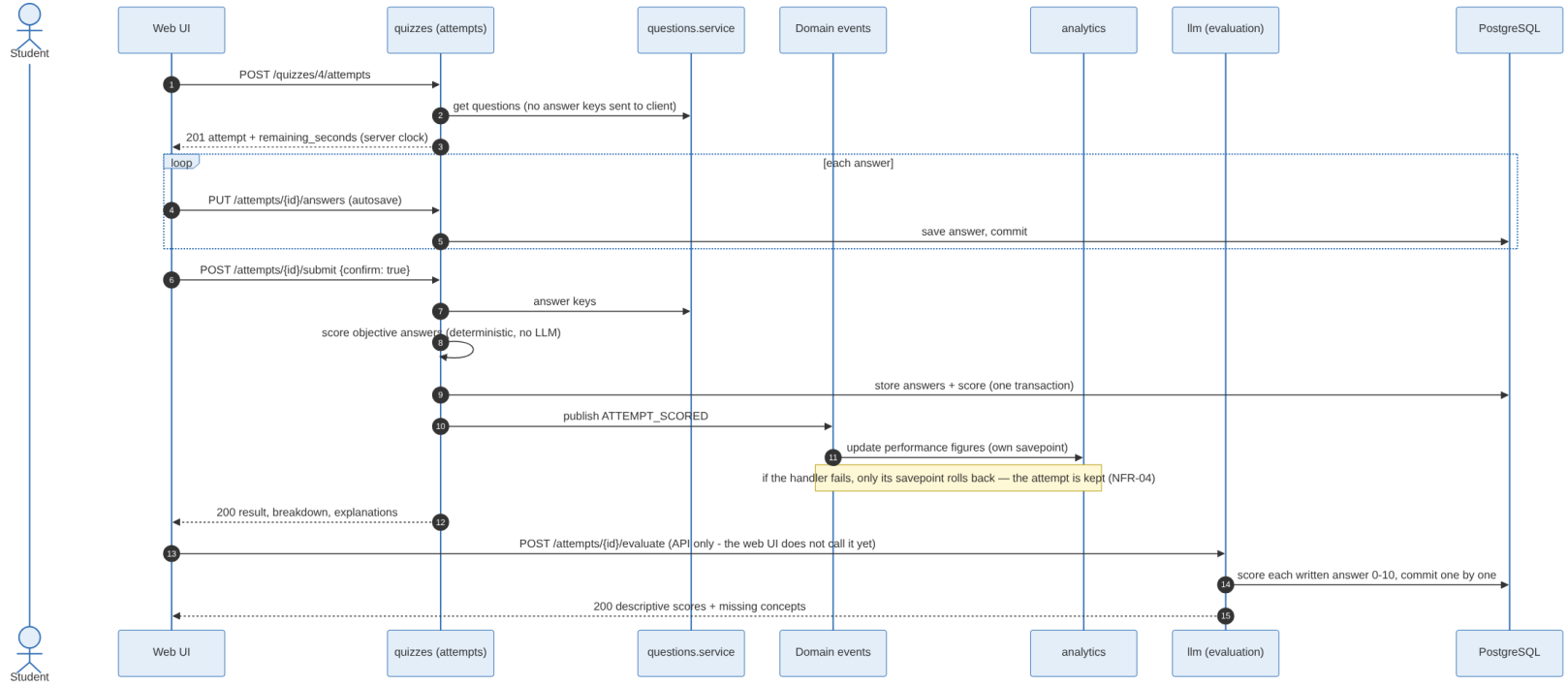


Figure 6: Sequence: attempt with autosave, deterministic scoring, domain event, written-answer evaluation

9.3 Generate questions with the LLM behind a review gate (FR-07, NFR-06)

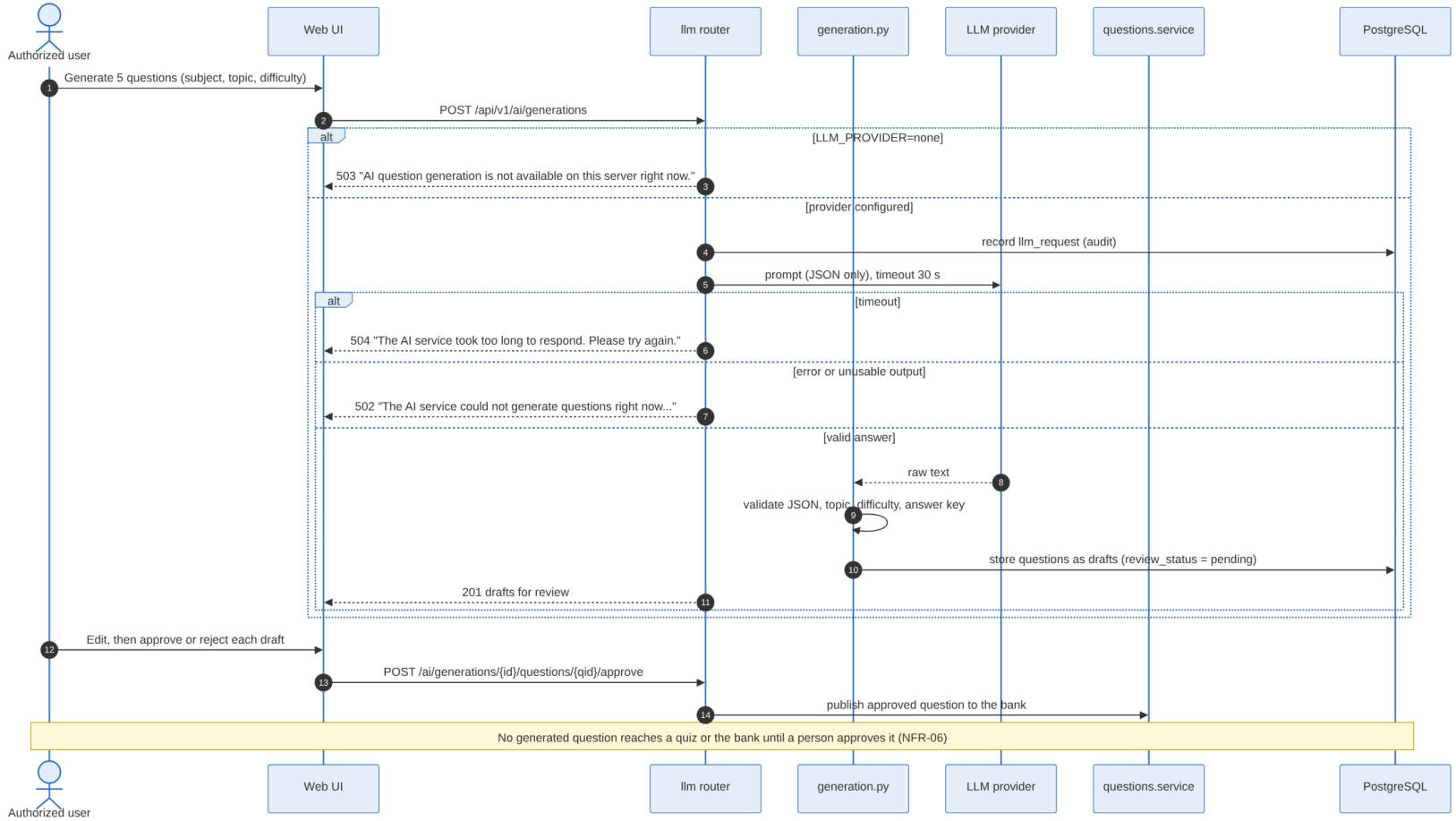


Figure 7: Sequence: LLM generation, failure handling and human approval

10. API design

10.1 Principles

- **Resource-oriented REST over HTTPS with JSON.** Nouns for resources (/channels, /materials, /quizzes, /attempts), HTTP verbs for actions; sub-resources for ownership (/channels/{id}/messages); a few explicit action endpoints where a state change is not a plain update (/quizzes/{id}/publish, /attempts/{id}/submit, /moderation/reports/{id}/remove).
- **Versioned.** Every route is under /api/v1. A breaking change gets a new prefix and a new contract file; v1 is never changed incompatibly.
- **Contract first-class.** The OpenAPI 3 document docs/api/openapi-v1.json is generated from the code, committed, and checked in CI: a pull request that changes the API without updating the contract fails.
- **Authentication.** `Authorization: Bearer <token>` on every route except /health, /accounts/register and /accounts/login. Authorization is declared per route by capability.
- **Requests and responses.** Request bodies are pydantic schemas with field-level validation. Responses are explicit schemas that never include password hashes, and never include answer keys before an attempt is submitted. List endpoints filter by query parameters (subject, topic, channel, difficulty, q). Files use multipart/form-data upload and are downloaded with Content-Disposition, nosniff and a restrictive CSP.
- **Status codes.** 201 for creation, 200 for reads and updates, 204 for deletes with no body, and 4xx/5xx as defined in section 11.

10.2 Endpoint groups

The v1 API has 108 operations on 91 paths. The groups, their main endpoints and the requirement each serves:

Group (router)	Ops	Main endpoints	FR
accounts	6	POST /accounts/register · POST /accounts/login · GET /accounts/me · GET /accounts/me/permissions	FR-01
profiles	6	GET/PATCH /profiles/me · PUT/DELETE /profiles/me/picture · GET /profiles/{user_id} · GET /profiles/{user_id}/picture	FR-01
channels	15	POST/GET /channels · GET /channels/mine · GET /channels/requests · POST /channels/{id}/approve · /reject · /join · /leave · GET /channels/{id}/members · DELETE .../members/{user_id} · posts (CRUD)	FR-02, FR-03
materials	12	POST/GET /materials · GET/PATCH/DELETE /materials/{id} · GET .../preview · GET .../download · PUT .../review · GET .../reviews · PUT/DELETE .../helpful · POST .../reports	FR-03, FR-04
questions	5	POST/GET /questions · GET /questions/{id} · PUT /questions/{id}/rating · POST /questions/{id}/reports	FR-03, FR-05
quizzes	5	POST /quizzes/preview · POST/GET /quizzes · GET /quizzes/{id} · POST /quizzes/{id}/publish	FR-05
quiz attempts	5	POST /quizzes/{id}/attempts · GET /attempts · GET /attempts/{id} · PUT /attempts/{id}/answers · POST /attempts/{id}/submit	FR-06
ai (generation, adaptive, AI reports)	11	POST/GET /ai/generations · PUT .../questions/{qid} · POST .../approve · /reject · POST .../quiz · POST /ai/adaptive-quizzes · POST .../answers · POST /ai/responses/{id}/reports	FR-07, FR-09
descriptive evaluation	2	POST /evaluations/descriptive · POST /attempts/{id}/evaluate	FR-08
ask ai	2	POST /ai/ask · POST /ai/ask/{id}/start (turn the answer into a quiz)	FR-09
chat	3	POST/GET /channels/{id}/messages · GET .../messages/{mid}/replies	FR-10

Group (router)	Ops	Main endpoints	FR
direct messages	9	GET/POST /conversations · GET /conversations/{id} · GET/POST .../messages · POST .../messages/{mid}/reports · GET /blocks · PUT/DELETE /blocks/{user_id}	FR-11
analytics	4	GET /analytics/dashboard · GET /analytics/recommendations · POST /analytics/recommendations/opened · GET /analytics/reconciliation	FR-12
reputation	3	GET /leaderboard · GET /reputation/me · GET /badges	FR-13
moderation	5	GET /moderation/queue · GET /moderation/reports/mine · GET /moderation/reports/{id} · POST .../remove · POST .../dismiss	FR-13
notifications	6	GET /notifications · GET /notifications/unread-count · GET/PUT /notifications/preferences · POST /notifications/read-all · POST /notifications/{id}/read	FR-14
admin	8	GET /admin/dashboard · GET /admin/users · POST /admin/users/{id}/suspend · /reinstate · GET /admin/channel-requests · GET /admin/reports · POST /admin/content/{type}/{id}/remove · GET /admin/audit-log	FR-14
operations	1	GET /health (no authentication, no internal details)	-

10.3 Example

```
POST /api/v1/attempts/42/submit   {"confirm": true}
```

```
200 {"id": 42, "status": "submitted", "result": {"display": "2/3 (66.7%)", "correct": 2, "incorrect": 1,
"breakdown": {...}}, "questions": [...]}
```

11. Error handling protocols

11.1 Error response format

Every error response is JSON with a human-readable `detail` string that the web UI shows as is. Validation errors add a per-field list:

```
422 {"detail": "Enter a valid email address", "errors": [{"field": "email", "message": "Enter a valid email address"}]}
```

Messages are written for users and match the Test Plan's expected wording (for example "Invalid username or password", "Unauthorized", "Only authorized users/admins can create Authorized Channels"). Responses never contain stack traces, SQL or configuration.

11.2 Status code catalogue

Code	Meaning in StudyHub	Example
400	The request is well-formed but not allowed as sent	"Use Leave Channel to leave a channel yourself"; "Confirm removal of this member"
401	No token, forged or expired token, or wrong login (sends WWW-Authenticate: Bearer)	"Not authenticated"; "Invalid username or password"
403	Authenticated but lacking the capability or membership	"Unauthorized"; a student creating an Authorized channel
404	Not found, or not visible to this user (existence is not revealed)	A pending channel opened by a non-member
409	Conflict with existing state	Duplicate username; "This attempt has already been submitted"; submit with unanswered questions before confirm: true
413 / 415	Upload too large (over 20 MB) or of a type not allowed	An .exe renamed to .pdf (magic bytes do not match)
422	Request body or parameters fail validation	Missing correct answer on a new question; blank chat message
502 / 504	LLM provider failed or timed out (30 s)	"The AI service took too long to respond. Please try again."
503	Feature unavailable or a transient database failure; nothing was saved	"AI question generation is not available on this server right now."; "Temporary problem saving your data..."

11.3 Server-side protocol

- **One transaction per request.** get_db opens a session per request and rolls it back on any exception, so a failed request never leaves partial data.
- **Transient database failures** (lost connection, lock timeout) are caught globally and returned as 503. The client may simply retry, and work committed earlier, such as autosaved answers, is kept (NFR-04).
- **Domain-event handlers run in their own savepoint.** If a reaction such as an analytics update or a notification fails, only its writes are undone. The triggering change, for example a submitted attempt, still commits, and analytics can reconcile later.
- **External calls fail fast and clearly.** LLM calls have a 30 s timeout and are recorded in llm_requests with their outcome. Invalid LLM output is rejected before anything is stored. Ask AI and generation report 502/503/504, and the rest of the platform is unaffected.
- **Overload.** RequestLimit admits at most as many API requests as there are database connections; the rest wait in arrival order instead of failing (NFR-01).

11.4 Client-side protocol

- api.js converts any non-2xx response into an ApiError carrying detail and the field errors; pages show the message next to the form or action that caused it.
- A 401 while signed in clears the stored token and raises a studyhub:signed-out event, which returns the user to the login page.
- Quiz answers are autosaved one by one, so a network error loses at most the answer being typed; the timer is owned by the server, so a reload resumes the same attempt.

12. Security architecture

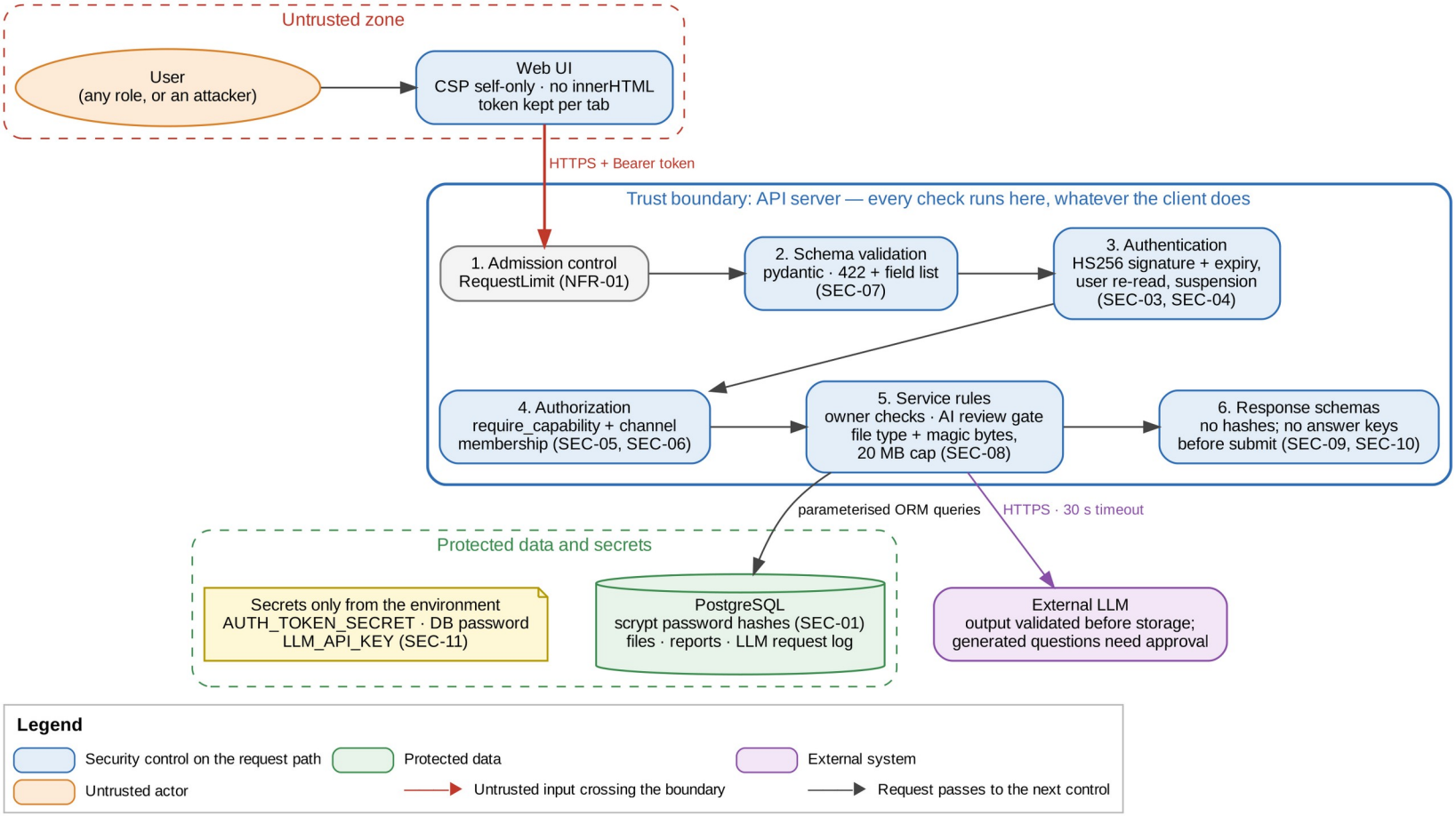


Figure 8: Security architecture: trust boundary and the controls on every request

The browser and everything it sends are untrusted. Every request crosses one trust boundary, the API server, and passes six controls in order: admission control, schema validation, authentication, authorization, service rules and response filtering. Hiding a button in the UI is never what makes an action safe, because every protected route declares the capability it needs. Secrets come only from the environment, and passwords are stored only as salted scrypt hashes.

Requirement	Control (component)	Status
SEC-01 Passwords hashed, minimum 8 characters	scrypt with per-password salt (app/core/security.py); length validated at registration	Implemented
SEC-02 At most 5 failed logins per account in 15 minutes	Not built; login returns the same 401 for every failure and uses a decoy hash against user enumeration	Deviation (16)
SEC-03 Tokens signed with a secret of at least 32 bytes, short-lived	HS256 with constant-time signature check, 30 min expiry, user re-read on every request; secret length is not checked at start-up	Partly (16)
SEC-04 Every endpoint except health, register and login needs a token	current_user dependency; a test walks all 102 protected routes with bad tokens (1,020 calls, all refused)	Implemented
SEC-05 Roles enforced on the server; no self-promotion	Capabilities per role (accounts/permissions.py), require_capability on routes; role ignored in registration input	Implemented
SEC-06 Membership-based access to channel content	channels.service visibility and membership checks; 404 hides non-visible channels	Implemented
SEC-07 All input validated; no stored script	Pydantic schemas; Safe DOM Builder (no innerHTML); CSP self-only on /app/	Implemented
SEC-08 Safe uploads	Extension allow-list confirmed by magic bytes; 20 MB cap; nosniff and CSP on downloads	Implemented
SEC-09 No answer keys before submission	Attempt schemas omit correct answers until the attempt is submitted	Implemented
SEC-10 Minimal data in responses	Explicit response schemas; email only to the owner and admins; no password hash ever	Implemented
SEC-11 Secrets out of the repository	Environment variables only, .env git-ignored, repository hygiene test scans tracked files	Implemented
SEC-12 One rating per user per item	Unique (item, user) constraints; a second rating updates the first	Implemented
SEC-13 Dependency scan on every pull request	Not yet in the CI workflow	Deviation (16)

13. Deployment view

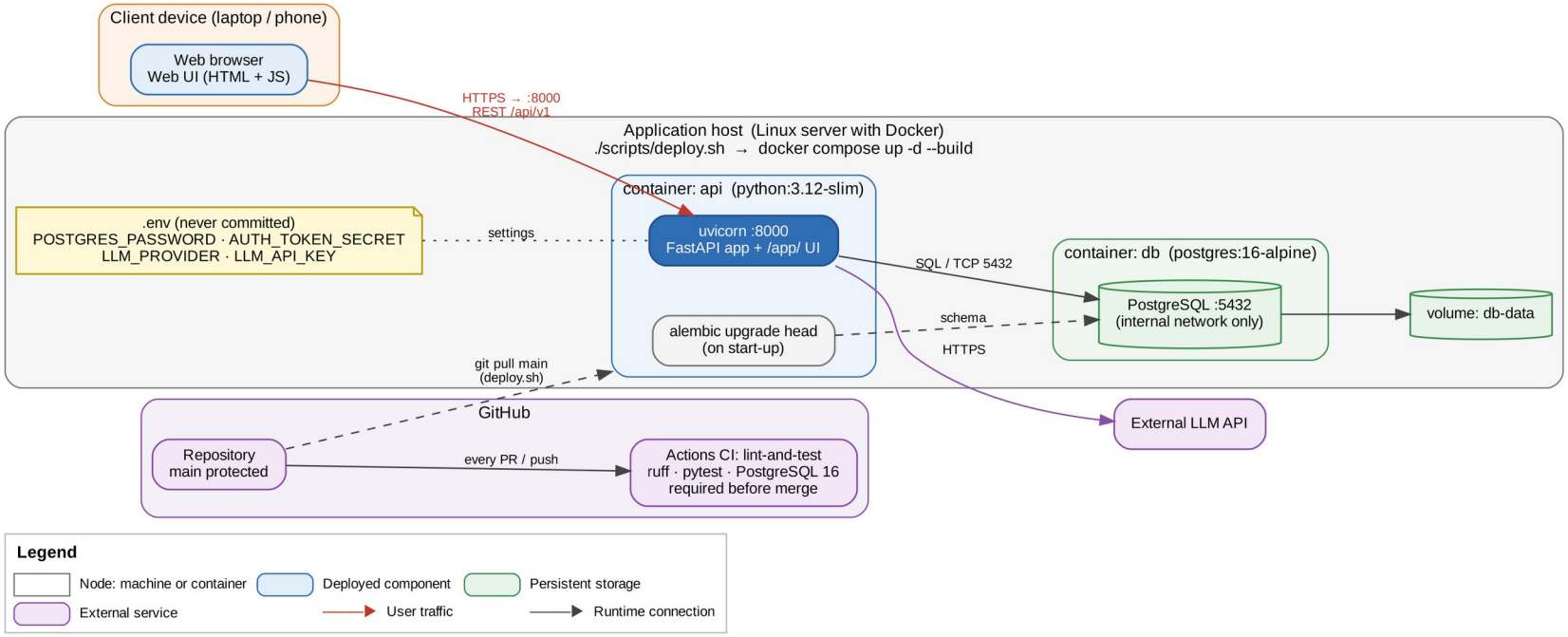


Figure 9: Deployment: client, application host, containers, storage and CI/CD

Node	Description
Client device	Any modern browser; nothing to install. The UI is served by the platform at /app/.
Application host	A Linux server with Docker. scripts/deploy.sh pulls main and runs docker compose up -d --build. Secrets are read from a .env file on the host that is never committed.
api container	python:3.12-slim. Runs alembic upgrade head at start-up, then uvicorn on port 8000 serving both the API and the web UI.
db container	postgres:16-alpine on the internal Docker network only, with a health check; data on the db-data volume, so it survives restarts and redeploys.
GitHub	Repository with protected main. GitHub Actions job lint-and-test runs ruff and the full pytest suite, including migration tests against a PostgreSQL 16 service, on every pull request and push.
External LLM API	Reached over HTTPS from the llm module only, with LLM_API_KEY from the environment. With LLM_PROVIDER=none or fake, no external call is made.

14. Quality attributes

NFR	Architectural mechanisms	Evidence (automated tests, docs/test-execution-log.md)
NFR-01 Performance	Stateless API, indexes on hot paths, RequestLimit admission control, in-process scoring	50 concurrent users: page p95 0.67 s (SQLite) and 1.11 s (PostgreSQL) against a 2 s target; bursts larger than the pool do not fail
NFR-02 Security	Section 12	Hashed passwords; 1,020 bad-token calls on 102 protected routes all refused; role and membership negative tests
NFR-03 Usability	Dashboard landing, one-click navigation, SVG charts, messages in plain language	Core flows reachable within three clicks; analytics shown as charts (browser tests)
NFR-04 Reliability	Transactions per request, autosave, savepoint-isolated event handlers, 503 on transient failure	No committed attempt or upload lost under simulated failures; recovery to a consistent state
NFR-05 Scalability	Constant work per request, indexed queries, stateless replicas, extractable modules	With 10x data, 28 hot paths stay under 1 s and the query count per request stays flat
NFR-06 LLM reliability	Review gate, output validation, AI-answer reporting, request log, scoring without LLM	Generated questions never published without approval; reports reach moderation; scores identical with LLM off
NFR-07 Maintainability	Module boundaries, generated interface docs, OpenAPI contract, migrations	A change to chat needs no change to quiz code; boundary and contract tests in CI

Test totals at the end of Sprint 2 (docs/unit-test-report.docx): 832 automated tests, 97.2% line coverage and 88.7% branch coverage of the app package.

15. Traceability

15.1 Functional requirements to design and tests

Requirement	Modules	API	UI screens	Test Plan cases
FR-01 Accounts, authentication, profile	accounts, profiles, reputation	/accounts, /profiles	Register, Log in, Profile	UT_01, UT_02 IT_01, IT_02 ST_01, ST_02
FR-02 Channel creation, classification, membership	channels	/channels	Channels, My Channels	UT_03, UT_04 IT_03, IT_04 ST_03, ST_04
FR-03 Authorized vs Normal channel content	channels, materials, questions	/channels, /materials, /questions	Channel page, Materials	UT_05, UT_06 IT_05, IT_06 ST_05, ST_06
FR-04 Material sharing, search, rating, reporting	materials, moderation	/materials	Materials, material page	UT_07, UT_08 IT_07, IT_08 ST_07, ST_08
FR-05 Question bank and peer quiz creation	questions, quizzes	/questions, /quizzes	Question bank, Quizzes	UT_09, UT_10 IT_09, IT_10 ST_09, ST_10
FR-06 Quiz attempt and automated evaluation	quizzes	/quizzes/{id}/attempts, /attempts	Attempt, Result, Quiz History	UT_11, UT_12 IT_11, IT_12 ST_11, ST_12
FR-07 LLM quiz generation and adaptive difficulty	llm, questions, quizzes	/ai/generations, /ai/adaptive-quizzes	Ask AI, Review drafts	UT_13, UT_14 IT_13, IT_14 ST_13, ST_14
FR-08 LLM descriptive answer evaluation	llm, quizzes	/evaluations/descriptive, /attempts/{id}/evaluate	Result (evaluation)	UT_15, UT_16 IT_15, IT_16 ST_15, ST_16
FR-09 LLM learning assistance, channel-scoped AI	llm, channels, questions	/ai/ask	Ask AI	UT_17, UT_18 IT_17, IT_18 ST_17, ST_18
FR-10 Channel chat	chat, notifications	/channels/{id}/messages	Channel page (chat)	UT_19, UT_20 IT_19, IT_20 ST_19, ST_20
FR-11 Direct chat and in-chat sharing	chat, moderation	/conversations	Messages	UT_21, UT_22 IT_21, IT_22 ST_21, ST_22
FR-12 Performance analysis and recommendations	analytics	/analytics	Performance, Recommended, Dashboard	UT_23, UT_24 IT_23, IT_24 ST_23, ST_24

Requirement	Modules	API	UI screens	Test Plan cases
FR-13 Peer reputation and content moderation	reputation, moderation, moderation_queue	/leaderboard, /reputation, /moderation	Leaderboard, Moderation	UT_25, UT_26 IT_25, IT_26 ST_25, ST_26
FR-14 Notifications and admin management	notifications, admin	/notifications, /admin	Notifications, Admin console	UT_27, UT_28 IT_27, IT_28 ST_27, ST_28

Every Test Plan case is an automated test whose name contains its case ID; the updated test case document (docs/test-cases-updated.docx) lists the test and its result for all 84 cases. Non-functional requirements trace to mechanisms and evidence in section 14, and security requirements to controls in section 12.

16. Deviations from the SRS and from SAD version 1.0

Item	Planned	As built	Reason / impact
Number of modules	11 modules (SAD 1.0)	14: profiles, admin and moderation_queue split out	Kept each module cohesive as features grew; no requirement affected.
Analytics data	No tables, derived on request	Stores derived figures (performance_attempts, performance_topics) updated by an event, rebuildable from attempts	Needed for NFR-01 and NFR-05 response times at scale.
File storage	Docker volume for uploads	Files and pictures stored in PostgreSQL tables (deferred loading)	Every API instance serves the same files and nothing is lost on redeploy (NFR-04, NFR-05).
Module integration	Direct service calls only	Direct calls plus in-process domain events	Avoids dependency cycles (quizzes to analytics, notifications); keeps attempts safe if a reaction fails.
Written-answer scoring in the UI	Scored after submission (FR-08)	API POST /attempts/{id}/evaluate works; the web UI does not call it yet, so answers show "Waiting for evaluation"	Small UI change remaining; demo triggers it through the API.
Adaptive quiz screen	Web screen for adaptive quizzes (FR-07)	Fully working API; no dedicated web screen	Time; demonstrated through the API.
Login lockout (SEC-02)	5 failed logins in 15 minutes	Not implemented	Open security item; uniform 401s and decoy hashing still limit enumeration.
Token secret length (SEC-03)	At least 32 bytes, checked	Required but its length is not validated at start-up	Open hardening item.
Dependency scan (SEC-13)	Scan on every pull request	Not in the CI workflow	Open item.
Test data	Password Pass@123 in the Test Plan	Pass@1234	The platform requires at least 8 characters (SEC-01).
Test execution	Manual execution of test steps on screens	All 84 cases automated against the API; screens covered by browser tests	Repeatable in CI; same expected messages.
LLM in the demo	Real LLM provider	Built-in offline provider (LLM_PROVIDER=fake) in demos	No API key available; the provider is switchable by configuration only.

Appendix A. Module dependency matrix

Each row lists the modules whose public service.py that module may call (app/modules/__init__.py, ALLOWED_DEPENDENCIES). Domain events (section 7.3) are not dependencies.

Module	May call	Subscribes to events
accounts	none	-
channels	accounts, moderation, notifications	-
materials	accounts, channels, moderation, reputation	-
questions	accounts, channels, moderation, reputation	-
quizzes	accounts, channels, questions	-
llm	accounts, channels, moderation, questions, quizzes	-
chat	accounts, channels, materials, moderation, questions, quizzes	-
analytics	accounts, channels, materials, questions, quizzes	ATTEMPT_SCORED (quizzes)
moderation	none	-
moderation_queue	accounts, channels, llm, materials, moderation, questions	-
reputation	accounts	-
notifications	accounts	CHANNEL_MESSAGE_POSTED (chat), QUIZ_PUBLISHED (quizzes), CHANNEL_POST_CREATED (channels)
profiles	accounts, materials, questions, quizzes, reputation	-
admin	accounts, channels, materials, moderation, notifications, questions	-